

[CS230] Foundations of Neural Networks: Cost Function & Gradient Descent

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1. Deep Learning Big Picture - *Completed*

Chapter 2. Logistic Regression as a Neural Network

- 2.1 Architecture & Forward Propagation - *Completed*
- **2.2 Cost Function & Gradient Descent (Current Unit)**
 - * Overview: Loss vs. Cost
 - * The Engine: Gradient Descent
 - * Why Log Loss? (Convexity)
 - * Implementation

Chapter 3. Shallow Neural Networks - *Upcoming*

지난 시간 복습 및 연결

우리는 지난 시간에 로지스틱 회귀의 '뇌 구조(Architecture)'를 만들고, 입력 신호를 흘려보내는 '순전파(Forward Propagation)'를 설계했습니다. 하지만 지금 이 신경망은 갓 태어난 아기와 같습니다. 세상에 대해 아무것도 모르죠(파라미터가 초기화된 상태). 이제 이 아이를 가르칠 시간입니다. 학습이란 "내가 얼마나 틀렸는지 확인하고(Cost), 고쳐 나가는(Gradient Descent) 과정"입니다.

1 Unit Overview

□ 핵심 목표

이 유닛은 머신러닝의 '엔진(Engine)'을 다룹니다. 차체(모델 구조)가 좋아도 엔진(학습 알고리즘)이 없으면 움직이지 않습니다.

- **구분:** 데이터 하나에 대한 오차(Loss)와 전체 성적표(Cost)를 구분합니다.
- **이유:** 왜 MSE 대신 **Log Loss(Binary Cross-Entropy)**를 써야 하는지 '지형(Topology)' 관점에서 이해합니다.
- **원리:** 산에서 내려오는 방법인 **경사 하강법(Gradient Descent)**의 원리를 배웁니다.
- **조절:** 학습률(Learning Rate)이 학습 속도에 미치는 영향을 분석합니다.

2 Essential Terminology

용어	기호	한 줄 핵심 요약
손실 함수 (Loss)	$L(\hat{y}, y)$	데이터 샘플 1개에 대한 오차 (작을수록 좋음)
비용 함수 (Cost)	$J(w, b)$	전체 학습 데이터 (m 개)에 대한 Loss의 평균
볼록성 (Convexity)	-	밥그릇처럼 매끄러운 모양 (최소점이 하나뿐인 안전한 지형)
기울기 (Gradient)	dw, db	현재 위치에서 가장 가파른 경사의 방향
학습률 (Learning Rate)	α	한 번 업데이트할 때 이동하는 보폭의 크기

3 Core Concepts: 학습의 매커니즘

3.1 1. Loss vs. Cost (오차의 정의)

한 줄 요약: Loss는 '쪽지시험 점수', Cost는 '학기말 평균 성적'입니다.

□ 시험 점수 비유 (직관적 비유)

- **Loss Function (L):** 1번 학생이 문제를 틀렸습니다. 이 학생 하나의 오차입니다.
- **Cost Function (J):** 우리 반 30명 전체의 평균 오차입니다. 선생님(모델)의 목표는 특정 학생만 잘 가르치는 게 아니라, 반 전체의 평균 성적(J)을 좋게 만드는 것입니다.

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

3.2 2. Why Log Loss? (MSE의 함정)

한 줄 요약: 로지스틱 회귀에서 MSE를 쓰면 함정이 많은 산이 되지만, Log Loss를 쓰면 매끄러운 밥그릇이 됩니다.

기술적 정의: 선형 회귀와 달리 Sigmoid 함수가 포함된 로지스틱 회귀에 $MSE(\frac{1}{2}(\hat{y} - y)^2)$ 를 적용하면 비용 함수가 비볼록(Non-Convex) 형태가 됩니다. 이는 수많은 국소 최적해(Local Optima)를 만듭니다.

MSE를 쓰면 안 되는 이유 (핵심 주의사항)

위 그림의 오른쪽(Non-Convex)을 보세요. 울퉁불퉁한 지형에서는 구슬을 굴렸을 때 가장 깊은 바닥(Global Minimum)이 아니라, 중간에 있는 작은 웅덩이(Local Minimum)에 갇혀버립니다. 학습이 망했다는 뜻입니다. 반면, 로그 손실(Log Loss)을 사용하면 왼쪽(Convex)처럼 매끄러운 그릇 모양이 되어, 어디서 시작하든 바닥으로 수렴합니다.

우리가 사용할 공식 (Binary Cross-Entropy):

$$L(\hat{y}, y) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

- 정답(y)이 1일 때: 예측($\hat{y}$)이 1이면 비용 0, 0이면 비용 ∞ .
- 틀렸을 때 무한대의 벌점을 주어 빠르게 고치도록 유도합니다.

3.3 3. Gradient Descent (경사 하강법)

한 줄 요약: 눈을 가린 채 산에서 가장 낮은 골짜기로 내려가는 방법입니다.

□ 안개 낀 산 하산하기 (직관적 비유)

당신은 짙은 안개가 낀 산 정상에 서 있습니다. 앞이 보이지 않습니다. 가장 낮은 곳(비용 최소화 지점)으로 가려면 어떻게 해야 할까요?

1. 발로 땅을 더듬어 경사가 가장 급하게 내려가는 방향을 찾습니다. (**Gradient 계산**)
2. 그 방향으로 한 발자국 내딛습니다. (**Update**)
3. 바닥에 도착할 때까지 반복합니다.

업데이트 공식:

$$w := w - \alpha \frac{\partial J}{\partial w}$$
$$b := b - \alpha \frac{\partial J}{\partial b}$$

- **빼기(-)의 의미:** 기울기가 양수(오르막)라면 w 를 줄여야(왼쪽으로 가야) 내려갈 수 있습니다. 반대 방향으로 가야 하므로 빼입니다.
- α (**Learning Rate**): 한 발자국의 크기입니다.

4 Practical Scenario: 학습률 α 의 중요성

학습률(Learning Rate) α 는 모델의 운명을 결정하는 가장 중요한 숫자(Hyperparameter)입니다.

- **Case A: α 가 너무 작을 때 (0.00001)**
개미처럼 기어갑니다. 해가 질 때까지(학습 종료까지) 산 중턱에도 못 갑니다. (수렴 속도 매우 느림)
- **Case B: α 가 너무 클 때 (10.0)**
거인의 점프입니다. 골짜기를 향해 뛰었는데 너무 멀리 뛰어서 반대편 산등성이에 처박힙니다. 오히려 더 높은 곳으로 올라갈 수도 있습니다. (**Overshooting / Divergence**)

5 Numerical Example: 비용 계산 해보기

□ 비용 함수 계산 실습 (실전 계산)

상황: 고양이 사진($y = 1$)을 보여줬는데, 모델이 0.8(80%)로 예측했습니다.

$$y = 1, \quad \hat{y} = 0.8$$

1. 손실(Loss) 계산: 공식: $L = -(1 \cdot \log(0.8) + 0 \cdot \log(0.2))$

$$L = -\log(0.8) \approx -(-0.223) = 0.223$$

상황 변경: 만약 모델이 0.1(10%)로 잘못 예측했다면?

$$L = -\log(0.1) \approx -(-2.30) = 2.30$$

→ 예측이 틀릴수록 벌 점(Loss)이 0.223에서 2.30으로 10배 넘게 커졌습니다! 이것이 Log Loss의 위력입니다.

6 Implementation (Python)

이론을 ‘numpy’ 코드로 옮겨봅시다.

```
1 import numpy as np
2
3 def compute_cost(A, Y):
4     """
5     A: 예측값 (1, m), Y: 실제값 (1, m)
6     """
7     m = Y.shape[1]
8
9     # log(0) 방지를 위해 아주 작은 값(epsilon)을 더해주는 것이 안전합니다.
10    epsilon = 1e-5
11
12    # Binary Cross-Entropy 수식 (Element-wise multiplication)
13    cost = -1/m * np.sum(Y * np.log(A + epsilon) + (1-Y) * np.log(1-A + epsilon))
14
15    return float(np.squeeze(cost)) # 배열을 스칼라로 변환
16
17 def update_parameters(w, b, dw, db, learning_rate):
18     """
19     경사 하강법 업데이트 단계
20     """
21     # 현재 위치에서 기울기(dw, db)의 반대 방향으로 만큼 alpha 이동
22     w = w - learning_rate * dw
23     b = b - learning_rate * db
24
25     return w, b
```

Listing 1: Cost Function and Optimization

7 FAQ: 초심자가 자주 묻는 질문

- Q1. 경사 하강법 공식에서 왜 더하지 않고 빼나요?
A. 기울기(Gradient)는 함수가 '증가하는' 방향을 가리킵니다. 우리는 비용을 '줄여야' 하므로 기울기의 반대 방향으로 가야 합니다. 그래서 빼니다.

- **Q2. 비용 함수가 0이 되면 좋은 건가요?**

A. 이론적으로는 완벽하지만, 현실에서는 **과적합(Overfitting)**을 의심해야 합니다. 문제 집 답을 달달 외운 상태일 수 있어서, 새로운 문제(Test Set)는 못 풀 수도 있습니다.

다음 단계 (Next Step)

이제 우리는 구조(**Architecture**)를 만들었고, 학습 방법(**Optimizer**)까지 장착했습니다. 다음 장에서는 이 모든 부품을 조립하여 실제 데이터를 입력받아 학습하고 예측하는 **전체 모델(Full Model)**을 완성하겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Cost Function:** 전체 데이터의 오차 평균(J)을 최소화하는 것이 목표다.
2. **Log Loss:** 로지스틱 회귀에는 MSE 대신 Log Loss를 써야 Convex(볼록)해진다.
3. **Gradient Descent:** $w_{new} = w_{old} - \alpha \cdot dw$. 경사를 타고 내려가는 알고리즘.
4. **Learning Rate:** 너무 크면 발산, 너무 작으면 느리다. 적절한 튜닝이 필요하다.